

Adversarial Search : Cutting Off Search

Introduction

Imagine playing a complex game like chess. As a human, you don't calculate every possible move until the very end of the game. Instead, you look a few moves ahead, evaluate the resulting board positions, and make a decision based on which position seems most favorable. This intuitive approach to game playing, where we stop analyzing at a certain depth and rely on an assessment of the current state, is precisely what artificial intelligence systems must emulate when faced with computationally intractable search spaces.

Traditional minimax search, while theoretically optimal, is impractical for games with vast state spaces because it requires exploring every possible game path to its conclusion. The sheer number of potential moves and counter-moves quickly leads to an exponential explosion of states, making a full search impossible even for powerful computers. This lecture introduces the critical concept of "minimax with cutoff," a pragmatic solution that allows AI agents to play complex games effectively by limiting search depth and employing sophisticated evaluation techniques. This approach is fundamental to developing intelligent game-playing agents, bridging the gap between theoretical optimality and real-world computational constraints.

Core Concepts

Term	Definition	Significance
Minimax with Cutoff	An adversarial search algorithm that limits the depth of its search tree, stopping at a predefined "cutoff" point rather than exploring to terminal game states.	Essential for making minimax computationally feasible in complex games by managing the exponential growth of the search space.
Evaluation Function	A heuristic function that assigns a numerical score to a non-terminal game state, estimating its value or favorability for a player.	Provides a way to assess the quality of a board position at the cutoff depth, allowing the algorithm to make decisions without reaching the game's end.

Term	Definition	Significance
Features	Measurable properties or characteristics of a game state (e.g., number of pieces, control of center, pawn structure).	Serve as the raw data inputs for an evaluation function, capturing relevant aspects of a board position that contribute to its strength.
Weights	Numerical coefficients assigned to each feature in an evaluation function, indicating its relative importance.	Determine how different features contribute to the overall score of a game state, allowing for fine-tuning the evaluation's accuracy.
Machine Learning	A field of AI focused on enabling systems to learn from data, often by adjusting internal parameters like weights, without explicit programming.	Provides the methodology for automatically determining optimal weights for evaluation functions, improving game-playing performance through experience.
Learning Rate	A hyperparameter in machine learning that controls the step size at which model weights are adjusted during optimization.	Dictates how quickly an AI agent adapts its evaluation function based on new experiences, balancing rapid learning with stability.
Self-Play	A training method where an AI agent plays against itself or a copy of itself to generate experience and improve its performance.	Crucial for learning effective evaluation functions in games, as it provides a continuous stream of data for weight adjustment without needing human opponents.
Backed-up Value	The value of a node determined by performing a deeper search (e.g., minimax) from that node and propagating the results back up the search tree.	Used as a more accurate "target" value during learning, helping to refine the evaluation function's prediction for a given state.

Detailed Analysis

1. The Necessity of Cutoff in Adversarial Search

In adversarial games, the goal is to find the optimal move by anticipating the opponent's best responses. The minimax algorithm achieves this by exploring the game tree, assuming both players play optimally. However, for games like chess, the game tree is astronomically large. A full search to the terminal states (win, loss, draw) is computationally impossible. This is where the concept of "minimax with cutoff" becomes indispensable.

Instead of searching to the very end, the algorithm stops at a predetermined depth (e.g., 4 plies, 8 plies). This "cutoff" point transforms the problem: at these artificial leaf nodes, the game hasn't ended, so there's no definitive win/loss/draw state. The challenge then becomes: how do we assign a value to these intermediate board positions? Without a way to evaluate these states, the minimax algorithm cannot back up meaningful values, rendering it ineffective. This necessitates the introduction of an evaluation function.

2. Designing Evaluation Functions for Partial States

An evaluation function is a heuristic that estimates the value of a non-terminal game state. It provides a numerical score, indicating how favorable a particular board position is for the current player. For simpler games, these functions can be relatively straightforward.

Consider Tic-Tac-Toe:

- * A simple evaluation function could count the number of rows, columns, or diagonals where the computer can still win, minus the number of rows, columns, or diagonals where the opponent can still win.

- * For example, if the computer (X) can win in 6 ways and the opponent (O) can win in 4 ways, the evaluation might be $6 - 4 = 2$. This suggests the computer's position is stronger.

This initial idea for an evaluation function often comes from human insight into the game – what the lecture refers to as coming "from my pocket." The designer observes the game, identifies key properties, and formulates a rule to quantify board strength.

3. Feature Engineering for Complex Games

For more complex games like chess, a single, simple rule is insufficient. Instead, evaluation functions are built upon multiple "features." Features are specific,

measurable properties of the board state that are believed to correlate with a player's advantage.

Examples of chess features include:

- * Number of queens a player has minus the opponent's queens.
- * Number of pawns near promotion.
- * Number of pieces under attack.
- * Total number of pieces for each player.
- * Number of bishops for each player.

Like the initial Tic-Tac-Toe example, these features are typically "engineered" by human experts who understand the game's nuances. They represent the domain knowledge distilled into quantifiable metrics.

4. Combining Features with Weighted Sums

Once multiple features are identified, they need to be combined into a single evaluation score. The most common approach is a weighted sum:

$$\text{Evaluation}(\text{state}) = w_1 * f_1(\text{state}) + w_2 * f_2(\text{state}) + \dots + w_m * f_m(\text{state})$$

Here, $f_i(\text{state})$ is the value of the i -th feature for a given board state, and w_i is the weight assigned to that feature. The weights reflect the relative importance of each feature. For instance, the difference in queen count might have a much higher weight than the difference in pawn count, as queens are generally more valuable. The challenge then shifts to determining these optimal weights.

5. Learning Weights through Machine Learning and Self-Play

Manually setting weights for dozens or hundreds of features is difficult and prone to error. This is where machine learning comes into play. The lecture highlights Arthur Samuel's checker-playing program from the 1950s as a pioneering example of using machine learning to learn these weights. Samuel is even credited with coining the term "machine learning."

The core idea is to learn the weights through experience, specifically through "self-play":

1. **Initialization:** Start with some initial, possibly random, weights for all features.
2. **Game Play:** Create two copies of the game-playing agent, A and B. Agent B's weights are fixed, while Agent A's weights will be updated.
3. **Evaluation and Backtracking:**

- * Agent A makes a move. At the current board position, it calculates an initial evaluation score using its current weights ($w_i * f_i$ summation).

- * Agent A then performs a deeper minimax search with cutoff (e.g., to depth 5), using its current evaluation function at the cutoff nodes.
- * The values from this deeper search are backed up to the initial node, yielding a "backed-up value." This backed-up value is considered a more accurate assessment of the board position's true worth.

4. **Weight Adjustment:**

- * Compare the initial evaluation score with the backed-up value. Let $\text{delta} = \text{backed-up value} - \text{initial evaluation}$.
- * If delta is positive (the board was better than initially thought), increase the weights of features that contributed positively to the initial evaluation and decrease the weights of features that contributed negatively.
- * If delta is negative (the board was worse than initially thought), decrease the weights of positive features and increase the weights of negative features.
- * The magnitude of this adjustment is controlled by a "learning rate," which is typically kept low to ensure stable learning over many games.
- * The update rule is conceptually $w_i = w_i + (\text{learning_rate} * \text{delta} * f_i)$.

This iterative process allows Agent A to refine its evaluation function. It learns to better predict the true value of board positions by comparing its shallow assessment with the results of a deeper look-ahead.

6. **Continuous Improvement and Robustness**

The learning process doesn't stop with one game. To ensure continuous improvement and prevent overfitting to a specific opponent (Agent B), several strategies are employed:

- * **Promoting the Learner:** Once Agent A significantly outperforms Agent B, Agent A's learned weights are copied to become the new fixed Agent B. A new Agent A is then initialized (perhaps with the new B's weights or slightly perturbed ones) to learn against this stronger opponent. This "bootstrapping" process ensures the agent always learns against a challenging opponent.
- * **Exploration:** Sometimes, the opponent (even a fixed B) might take random actions. This introduces stochasticity, making the learning agent more robust and less likely to exploit specific weaknesses of a deterministic opponent.
- * **End-Game Information:** While cutoff search is used, information from actual game outcomes (win/loss/draw) can also be used to refine weights, especially by backing up values from the very end of the game.

This iterative, self-improving cycle is a hallmark of reinforcement learning, a field closely related to this approach, where agents learn optimal behaviors through trial and error in an environment.

Key Takeaways

- [] Minimax search requires a cutoff depth for practical application in complex games due to computational limits.
- [] An evaluation function is essential to assign a heuristic value to non-terminal game states at the cutoff depth.
- [] Evaluation functions for complex games are typically constructed as a weighted sum of various "features" of the board state.
- [] Features are domain-specific properties, often initially identified by human experts, that contribute to a player's advantage.
- [] Machine learning, particularly through self-play, is used to automatically learn and optimize the weights assigned to each feature.
- [] Weight updates occur by comparing an initial evaluation of a state with a more accurate "backed-up value" obtained from a deeper search.
- [] The learning rate controls the magnitude of weight adjustments, influencing how quickly the agent adapts its evaluation function.
- [] Continuous improvement involves promoting a stronger learning agent (A) to become the new fixed opponent (B) for subsequent learning iterations.
- [] Modern approaches to game AI often involve machines learning features themselves using deep neural networks, moving beyond human-engineered features.

Conclusion

The journey from simple minimax to minimax with cutoff and learned evaluation functions marks a significant evolution in artificial intelligence for game playing. By acknowledging the computational limits of exhaustive search, AI researchers developed pragmatic solutions that leverage human domain knowledge (features) and the power of machine learning (weight optimization). This approach, pioneered by figures like Arthur Samuel, transformed game AI from rigid, rule-based systems into adaptive, learning agents.

The principles discussed here—using evaluation functions, feature engineering, weighted sums, and self-play for learning—form the bedrock of many successful game-playing AIs. Looking forward, this foundation directly connects to modern advancements in reinforcement learning and deep learning, where neural networks can automatically discover and learn features from raw game states, pushing the boundaries of what's possible in complex strategic environments. The ability to learn and adapt through experience, rather than being explicitly programmed for every scenario, remains a central theme in the ongoing quest for truly intelligent agents.